Invoice Automation: Edge Cases & Solutions

1. The Timezone Trap

The Problem: Your cron job is scheduled to run at 9:00 AM Server Time. If you deploy this website to the cloud (like Render, Heroku, or AWS), cloud servers run on UTC Time by default. 9:00 AM UTC is 2:30 PM IST.

The Solution: We can force node-cron to respect a specific timezone (like Asia/Kolkata) regardless of where the server lives by passing a timezone options object to the scheduler.

2. The Spam Filter (Rate Limiting)

The Problem: If you have 200 invoices due on the exact same day, your Express server will fire 200 rapid-fire requests to n8n in a single second. Gmail and other SMTP providers will likely flag your account for spam and temporarily block your email account.

The Solution: We can add a simple 2-second delay between each webhook call inside your Express loop so the emails trickle out naturally and mimic human sending behavior.

3. Invalid Client Email Addresses

The Problem: If you accidentally type a client's email wrong (e.g., john@gmail..com), the Express server will push it to n8n, n8n will try to email it, and Gmail will reject it. Express puts it in the Queue and tries 10 more times until it gives up. You have no way of knowing why it gave up unless you inspect the database.

The Solution: We could add a simple "Failed Reminders" tab to your React Dashboard so you can visually see which emails bounced and easily fix the email addresses from the UI.

4. Overdue Harassment (or Lack Thereof)

The Problem: Right now, the system elegantly sends a reminder 7 days before, and exactly on the due date. But what happens if a client ignores the email and is 14 days overdue? Currently, the automated system stops talking to them after the due date.

The Solution: We could update the daily sweep to search for invoices that are exactly 3 days overdue, 7 days overdue, etc., and send a slightly more "urgent" webhook payload to n8n to prompt them for payment.